- *Minimal APIs*—Simple HTTP APIs that can be consumed by mobile applications or browser-based single-page applications (SPAs)
- *Web APIs*—An alternative approach for building HTTP APIs that adds more structure and features than minimal APIs
- *gRPC APIs*—Used to build efficient binary APIs for server-to-server communication using the gRPC protocol
- *Razor Pages*—Used to build page-based server-rendered applications
- *MVC controllers*—Similar to Razor Pages; used for server-based applications but without the page-based paradigm
- *Blazor WebAssembly*—A browser-based SPA framework using the WebAssembly standard, similar to JavaScript frameworks such as Angular, React, and Vue
- *Blazor Server*—Used to build stateful applications, rendered on the server, that send UI events and page updates over WebSockets to provide the feel of a client-side SPA but with the ease of development of a server-rendered application

All these paradigms use the core functionality of ASP.NET Core and layer the additional functionality on top. Each paradigm is suited to a different style of web application or API, so some may fit better than others, depending on what sort of application you're building.

Traditional page-based, server-side-rendered web applications are the bread and butter of ASP.NET development, both in the previous version of ASP.NET and now in ASP.NET Core. The Razor Pages and MVC controller paradigms provide two slightly different styles for building these types of applications but have many of the same concepts, as you'll see in part 2.